

Core Spring Framework

2-Hour Training for Fresher Developers

Spring Framework fundamentals — explicitly excluding Spring Boot

Suggested 2-Hour Flow

Section	Time
Why Spring + IoC/DI	20 min
Container, Beans & Lifecycle	20 min
Configuration & Stereotypes	20 min
AOP	15 min
Spring JDBC / DAO	15 min
Transactions	15 min
Live Demo	10 min
Wrap-up / Q&A	5 min

Slide 1 — Core Spring Framework

Suggested delivery time: 3 minutes

- From Java Objects to a Managed Enterprise Application
- Learning objectives: Why Spring; IoC & Dependency Injection; Spring Container; bean lifecycle & scopes; configuration; stereotypes; AOP; Spring JDBC/DAO; transactions; how Spring Boot later simplifies these concepts.
- Scope: Spring Framework fundamentals only — no Spring Boot.

Slide 2 — Why Was Spring Introduced?

Suggested delivery time: 7 minutes

- Traditional Java classes often create their own dependencies, producing tight coupling and difficult testing.
- Problems: implementation replacement is harder; object creation is scattered; configuration and business logic become mixed.
- Spring lets a container create objects, resolve dependencies, inject them, and manage their lifecycle.
- Key message: Don't let every class create and manage its own dependencies.

Code snippet

```
public class OrderService {  
    private PaymentService paymentService =  
        new PaymentService();  
  
    public void placeOrder() {  
        paymentService.pay();  
    }  
}
```

Slide 3 — IoC & Dependency Injection

Suggested delivery time: 10 minutes

- IoC is the design principle where control over object creation and dependencies is transferred to the container.
- Dependency Injection is a primary technique Spring uses to implement IoC.
- Benefits: loose coupling, testability, replaceable implementations, cleaner architecture.

Code snippet

```
public interface PaymentService {
    void pay(double amount);
}

@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder(double amount) {
        paymentService.pay(amount);
    }
}
```

Slide 4 — Dependency Injection Types

Suggested delivery time: 10 minutes

- Constructor injection — recommended for mandatory dependencies: explicit, testable, supports final fields.
- Setter injection — useful when a dependency is optional or changeable.
- Field injection — concise but hides dependencies, complicates unit testing, and prevents final fields.
- Recommendation: constructor injection for mandatory dependencies.

Code snippet

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

@Service
public class ReportService {
    private EmailService emailService;

    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}

// Field injection:
@Autowired
private UserRepository userRepository;
```

Slide 5 — What Is a Spring Bean?

Suggested delivery time: 5 minutes

- A Spring Bean is an object created, configured, and managed by the Spring IoC container.
- Container responsibilities: instantiate beans, resolve and inject dependencies, manage lifecycle and scopes, apply post-processors, and create proxies when required.
- `new PaymentService()` creates an ordinary Java object; a container-managed `PaymentService` is a Spring Bean.

Code snippet

```
@Component
public class PaymentService {
    public void pay() {
        System.out.println("Payment processed");
    }
}
```

Slide 6 — BeanFactory vs ApplicationContext

Suggested delivery time: 8 minutes

- BeanFactory is the basic IoC container and provides core bean management.
- ApplicationContext is a more complete container with bean management plus events, internationalization, resource loading, and infrastructure integration.
- Practical rule: ApplicationContext is the normal choice for application development.

Code snippet

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);  
  
OrderService service =  
    context.getBean(OrderService.class);
```

Slide 7 — Bean Lifecycle

Suggested delivery time: 10 minutes

- Typical lifecycle: create bean → inject dependencies → post-processing → initialization callbacks → bean ready → application use → destruction callbacks.
- Callbacks include `@PostConstruct`, `@PreDestroy`, `InitializingBean`, `DisposableBean`, custom `initMethod` and `destroyMethod`, and `BeanPostProcessor`.
- Use lifecycle callbacks for initialization and cleanup of resources managed by a bean.

Code snippet

```
@Component
public class CacheManager {

    @PostConstruct
    public void initialize() {
        System.out.println("Loading cache");
    }

    @PreDestroy
    public void shutdown() {
        System.out.println("Clearing resources");
    }
}
```

Slide 8 — Bean Scopes

Suggested delivery time: 8 minutes

- Singleton: one instance per Spring container; this is the default.
- Prototype: a new instance when requested.
- Web scopes include request, session, application, and websocket.
- Important: singleton means one instance per Spring container, not necessarily one instance per JVM.

Code snippet

```
@Component
@Scope("prototype")
public class ReportGenerator {
}

// Assuming context is an ApplicationContext:
ReportGenerator r1 =
    context.getBean(ReportGenerator.class);

ReportGenerator r2 =
    context.getBean(ReportGenerator.class);

System.out.println(r1 == r2); // false
```

Slide 9 — Spring Configuration Approaches

Suggested delivery time: 10 minutes

- XML configuration: common in legacy Spring applications.
- Java configuration: `@Configuration` and `@Bean` provide type-safe configuration.
- Annotation configuration: `@ComponentScan` discovers stereotype-annotated classes.
- All approaches answer: which objects should Spring create, and how should they be connected?

Code snippet

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }

    @Bean
    public OrderService orderService(
        PaymentService paymentService) {
        return new OrderService(paymentService);
    }
}

// Annotation scanning:
@Configuration
@ComponentScan("com.example")
public class ScanConfig { }
```

Slide 10 — Spring Stereotype Annotations

Suggested delivery time: 8 minutes

- `@Component`: generic Spring-managed component.
- `@Service`: business/service layer.
- `@Repository`: persistence/DAO layer; also participates in persistence exception translation.
- `@Controller`: web/MVC layer.
- These annotations communicate component responsibility and make architecture easier to understand.

Code snippet

```
@Repository
public class CustomerRepository { }

@Service
public class CustomerService {
    private final CustomerRepository repository;

    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }
}

@Controller
public class CustomerController { }
```

Slide 11 — AOP: Why Do We Need It?

Suggested delivery time: 8 minutes

- Cross-cutting concerns appear across many classes: logging, security, transactions, auditing, performance monitoring, and caching.
- Without AOP, the same infrastructure code gets duplicated and business logic becomes polluted with technical concerns.
- AOP separates cross-cutting logic from core business logic.

Code snippet

```
public void createOrder(Order order) {  
    log.info("Starting");  
  
    // Business logic  
  
    log.info("Completed");  
}
```

Slide 12 — AOP Concepts & Advice

Suggested delivery time: 10 minutes

- Aspect: module containing cross-cutting logic.
- Join point: a point during execution where behavior can be applied.
- Pointcut: expression selecting join points.
- Advice: code executed at selected join points.
- Advice types: `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`.
- Spring AOP is primarily proxy-based, so self-invocation can bypass proxy-based advice.

Code snippet

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.service.*(..))")
    public void log(JoinPoint joinPoint) {
        System.out.println(
            "Calling: " +
            joinPoint.getSignature().getName());
    }
}

@Configuration
@EnableAspectJAutoProxy
@ComponentScan("com.example")
public class AppConfig { }
```

Slide 13 — Spring JDBC & DAO

Suggested delivery time: 12 minutes

- Traditional JDBC requires connection, statement, result-set, exception handling, and resource cleanup boilerplate.
- JdbcTemplate simplifies resource management, execution, and exception translation while allowing you to write SQL.
- Spring JDBC integrates with Spring transaction management.

Code snippet

```
@Repository
public class CustomerDao {
    private final JdbcTemplate jdbcTemplate;

    public CustomerDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public Customer findById(long id) {
        String sql =
            "SELECT id, name FROM customer WHERE id = ?";

        return jdbcTemplate.queryForObject(
            sql,
            (rs, rowNum) ->
                new Customer(
                    rs.getLong("id"),
                    rs.getString("name")),
            id);
    }
}

@Bean
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    return new JdbcTemplate(dataSource);
}
```

Slide 14 — Transaction Management

Suggested delivery time: 12 minutes

- Transactions keep a group of related database operations atomic.
- `@Transactional` is normally applied at the service/business-operation boundary.
- Spring uses a transaction manager and, commonly, a proxy around the target bean.
- Runtime exceptions generally trigger rollback by default; rollback rules can be customized.
- Self-invocation can bypass proxy interception.

Code snippet

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(Order order) {
        orderDao.insert(order);

        inventoryDao.reserve(
            order.getProductId(),
            order.getQuantity());

        paymentDao.createPayment(order);
    }
}

@Configuration
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public PlatformTransactionManager transactionManager(
        DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}
```

Slide 15 — Live Coding: Build a Mini Spring Application

Suggested delivery time: 10 minutes

- Demo goal: build Controller → Service → Repository and let Spring create and wire the objects.
- Ask: Who created OrderService? Who created OrderRepository? Who injected the repository?
- Expected answer: the Spring IoC container.

Code snippet

```
@Configuration
@ComponentScan("com.example")
public class AppConfig { }

@Repository
public class OrderRepository {
    public void save() {
        System.out.println("Saving order");
    }
}

@Service
public class OrderService {
    private final OrderRepository repository;

    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }

    public void createOrder() {
        repository.save();
        System.out.println("Order created");
    }
}

public class Application {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        OrderService service =
            context.getBean(OrderService.class);

        service.createOrder();
    }
}
```

Slide 16 — From Core Spring to Spring Boot + Q&A;

Suggested delivery time: 9 minutes

- Core Spring can require explicit configuration for component scanning, transaction management, AOP, DataSource, JdbcTemplate, and transaction managers.
- Spring Boot later uses dependencies, classpath detection, properties, conventions, and auto-configuration to reduce manual configuration.
- Final questions: What is IoC? What is DI? ApplicationContext vs BeanFactory? Why constructor injection? Why does @Transactional involve a proxy?
- Final takeaway: Spring is not magic — it is a container that manages objects, dependencies, lifecycle, and infrastructure around your application.